

INSuRE Fall 2022

Final Report – December 11, 2022

**Making Signals Naturally Difficult to Classify with Supervised Learning Algorithms
via Model-Agnostic Adversarial Perturbations**

Sponsor



Technical Director

Kyle McClintick - kyle.mcclintick@ll.mit.edu



**Erik Jonsson School of Engineering and Computer Science
Department of Computer Science
Team 1**

Team Members

Varun Potluri – varun.potluri@utdallas.edu
Raviteja Avuthu - raviteja.avuthu@utdallas.edu
Saipriya Nimmagadda - saipriya.nimmagadda@utdallas.edu
Raghuveer Pachipulusu - raghuveer.pachipulusu@utdallas.edu
Vijay Srinivas Panchangam - venkatabhavanavijaysrinivas.panchangam@utdallas.edu

Faculty Support

Brian Ricks - bwr031000@utdallas.edu

Table of Contents

1. Project Summary.....	2
1.1 Project Description.....	2
1.2 Motivation.....	2
2. Organization.....	3
2.1 Estimated Project Budget.....	3
2.2 Timeline of Milestones.....	3
3. Executive Summary.....	4
3.1 Impact on different trends.....	5
4. Previous Work.....	5
4.1 Evaluating adversarial evasion attacks in the context of wireless communication.....	5
4.2 Explaining and Harnessing Adversarial Examples.....	6
4.3 Query-efficient hard-label black-box attack: An optimization-based approach.....	6
5. Research Project Plan and Approach.....	6
5.1 Research Project Phases.....	6
5.2 Resources Required.....	6
6. Mid Term Report.....	6
6.1 Explaining Adversarial Examples:.....	6
6.2 Researched on different attack techniques:.....	7
6.3 Dataset Extraction and Analysis:.....	7
7. Final Report.....	8
7.1 Model Analysis.....	8
i. Previous Method.....	8
ii. Proposed Model.....	9
7.2 Our Model.....	13
7.3 Challenges Faced.....	14
8. Conclusion and Future Work.....	15
9. Investigator Biographies.....	15
10. Bibliography.....	16

1. Project Summary

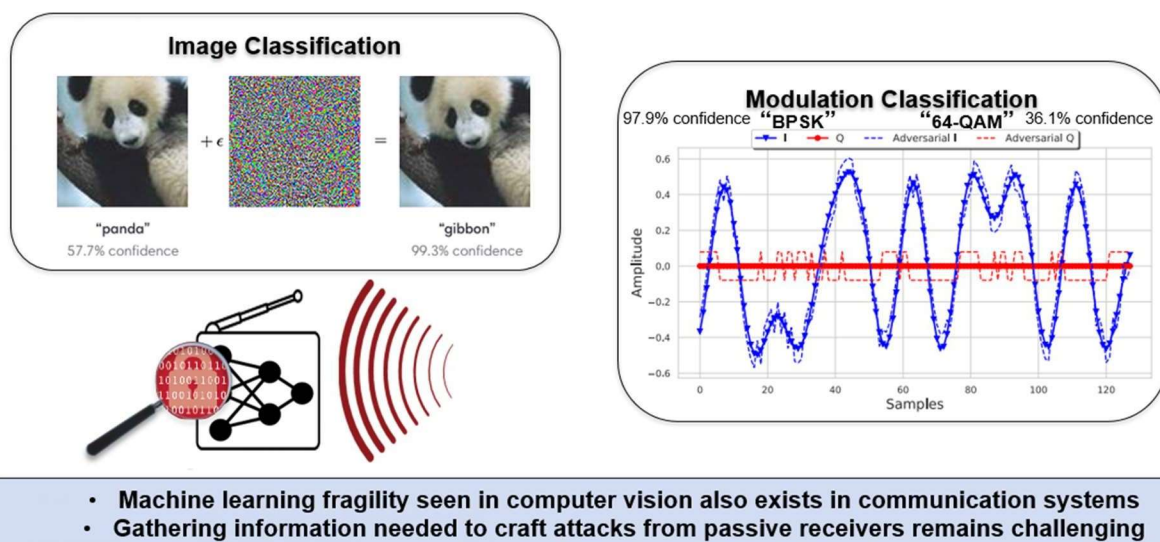
1.1 Project Description

Till now the attacks on classifiers were mostly through changing the data that is trained to get the wrong classifier which is the Data Poisoning or change the input during the testing, which is called the Data Evasion, also known to be the black box, grey box and the white box attacks respectively. Measures have been taken and models were evolved to overcome and identify the attackers. But zero query attacks are truly a stealthy attack. The attacker won't have any access to the victim system. The attacker can implement an attack plan once by perturbing multiple objects in each scene and submitting the perturbed input.

1.2 Motivation

Machine learning algorithms build a model based on sample data, known as training data, to make predictions or decisions without being explicitly programmed to do so. Machine learning algorithms are used in a wide variety of applications, such as in medicine, email filtering, speech recognition, and computer vision, where it is difficult or unfeasible to develop conventional algorithms to perform the needed tasks. And we are at that very basic step of training different kinds of Machine Learning models to predict the future outcomes. Well, you might have seen different attacking techniques on Machine Learning Models. The major of them can be classified into Data poisoning and data evasion. The main difference between data poisoning attacks and evasion attacks is the assumption of an attacker's ability. Adversarial Perturbations comes under the Data Evasion Attack. An attacker can target the data in different ways. The attacker can inject malware or false data at the time of testing phase i.e., giving modified input to model, is called Data Evasion. The attacker can inject malware or false data at the time of training which is called Data poisoning. This is the most widespread and old attack which is used on the data before sending the data to the model for training. So that it can classify the test data wrong. Till now the attacks on classifiers were mostly through changing the data that is trained to get the wrong classifier which is the Data Poisoning or change the input during the testing, which is called the Data Evasion, also known to be the black box, grey box, and the white box attacks respectively. Measures have been taken and models were evolved to overcome and identify the attackers. But zero query attacks are truly a stealthy attack. The attacker won't have any access to the victim system. The attacker can implement an attack plan once by perturbing multiple objects in each scene and submitting the perturbed input.

Let's start with the most basic question, what is an adversarial attack? It is a technique that attempts to fool models with deceptive data, that is it makes a malicious attempt that tries to fool the models with deceptive data. If we assume a datapoint x_0 (x zero) then the adversarial attack will try to perturb x_0 to a new data point x such that x is misclassified by the classifier. A false data is generated to deceive the classifiers. Such inputs are purposefully designed to cause ML models to make a mistake and change its prediction. One of the best ways to demonstrate an adversarial example is by showing the most famous panda example.



In this, the attack uses Fast Gradient Sign method, which will be explained later in detail. The first picture shows us an image of panda. X is the original input image, and it is correctly classified as “panda”. y is the ground truth label for x , θ represents the model parameters, and $J(\theta, x, y)$ is the loss that is used to train the network. The attack backpropagates the gradient back to the input data to calculate delta change in $J(\theta, x, y)$. Then, it adjusts the input data by a small step ϵ or 0.007 (as mentioned in the picture) in the direction that will maximize the loss. The resulting perturbed image, x' , is then misclassified by the target network as a “gibbon” when it is still clearly a “panda”. Machine learning fragility seen in computer vision also exists in communication systems. Gathering information needed to craft attacks from passive receivers remains challenging.



DARPA
DEFENSE ADVANCED
RESEARCH PROJECTS AGENCY

Radio Frequency Machine Learning Systems (RFMLS)
Mr. John Davies

Unclassified//FOUO

Cybersecurity Directorate – Top Challenges FY22
Unclassified Version – 7 March 2022

Primary:

Priority: 1

PoC: Cathie Cope, Neal Ziring

Title: (U) AI/ML in Cyber Defense

Description: (U) Two parts:

1. Detection of, prevention of, and response to malicious use of AI in cyber-attacks and identifying next-generation attack surfaces and infrastructure. How can we find and characterize and defeat adversaries when they employ AI/ML in their malicious cyber operations.
2. Use of AI/ML to improve, empower, accelerate defensive operations. Use of AI/ML to improve defensive workflow, or to enhance responses to attacks.



An increasing use of machine learning at the physical layer by DoD contractors presents a new attack surface

2. Organization

2.1 Estimated Project Budget

As our team does not anticipate any expenses, we do not have any project budget.

2.2 Timeline of Milestones

Key Deliverables	Due Date / Expected Completion Date
Project Proposal	September 9, 2022
Phase 1: Get a basic understanding of Adversarial Perturbations and fundamentals	September 14, 2022
Final Midterm Presentation	October 21, 2022
Phase 2: Know how to create a substitute model and how to craft adversarial samples	October 21, 2022
Phase 3: Attack the target model with these adversarial samples	November 11, 2022

Phase 4: Once we validate our attack model, then document this study and publish using some publication framework.	December 2, 2022
Final Report Presentation	December 9, 2022
Final Report	December 11, 2022

2.3 Specific Aims

Phase 1: Get a basic understanding of Adversarial Perturbations and fundamentals

Phase 2: Know how to create a substitute model and how to craft adversarial samples. We want to identify the most powerful attack that can use (most incorrect classifications for a given power constraint) to attack an unknown, unquarriable classifier.

Phase 3: After identifying the attack, we now validate our attack on RFML 2018 dataset forcing it to misclassify. Once we validate our attack model, then document this study for future work.

3. Executive Summary

Adversarial Perturbations is a form of attack in which bad actors attempt to manipulate machine learning models that detect attacks and anomalies by “poisoning” the data they train on. These Perturbations give a understanding about the model to the attackers so that they can change the data to make the classifiers go wrong. Black box attacks, White box attacks and grey box attacks mainly target on the data to make the classifier go wrong. But the no box attacks target the meta data. We think that with experimentation and research; we could attempt to test an existing open-source dataset and model against existing no box attack or query-based attacks and to help inform the maintainers of the dangers that such techniques may face and how to mitigate them.

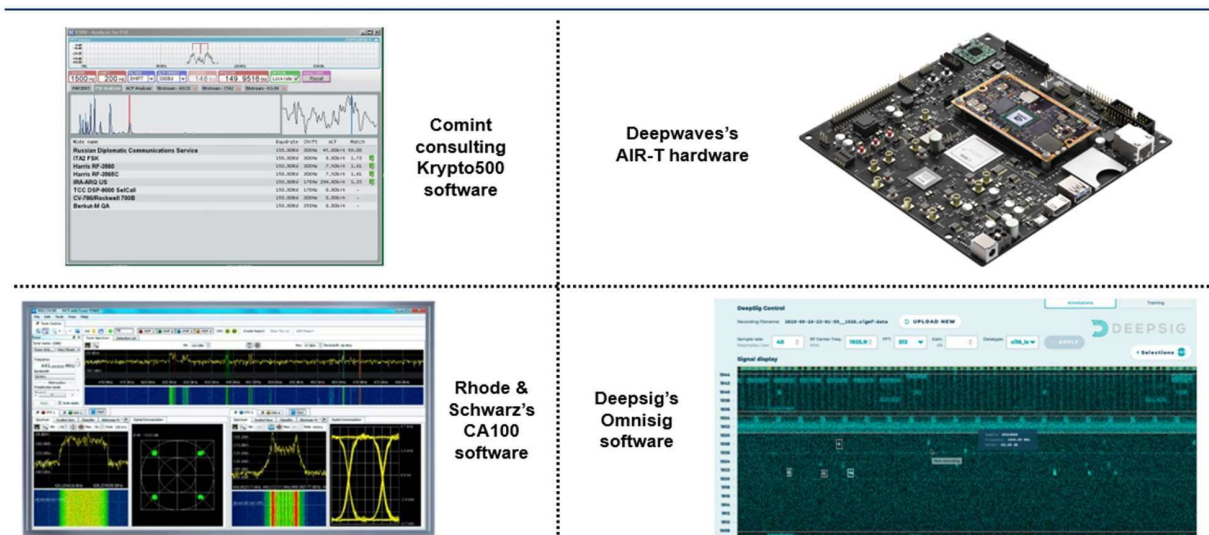
In many problems, the precision of an individual input feature is limited. For example, digital images often use only 8 bits per pixel, so they discard all information below $1/255$ of the dynamic range. Because the precision of the features is limited, it is not rational for the classifier to respond differently to an input x than to an adversarial input $\tilde{x} = x + \eta$ if every element of the perturbation η is smaller than the precision of the features. we expect the classifier to assign the same class to x and $\tilde{x}$ so long as $\|\eta\|_\infty < \epsilon$, where ϵ is small enough to be discarded by the sensor or data storage apparatus associated with our problem. Consider the dot product between a weight vector w and an adversarial example $\tilde{x}$: $w \cdot \tilde{x} = w \cdot x + w \cdot \eta$. The adversarial perturbation causes the activation to grow by $w \cdot \eta$. If w has n dimensions and the average magnitude of an element of the weight vector is m , then the activation will grow by mn . Since $\|\eta\|_\infty$ does not grow with the dimensionality of the problem but the change in activation caused by perturbation by η can grow linearly with n , then for high dimensional problems, we can make many infinitesimal changes to the input that add up to one large change to the output.

This explanation shows that a simple linear model can have adversarial examples if its input has sufficient dimensionality them. LSTMs, ReLUs etc are all intentionally designed to behave in very linear ways, so that they are easier to optimize. More nonlinear models such as sigmoid networks are carefully tuned to spend most of their time in the non-saturating, more linear regime for the same reason. This linear behavior suggests that cheap, analytical perturbations of a linear model should also damage neural networks.

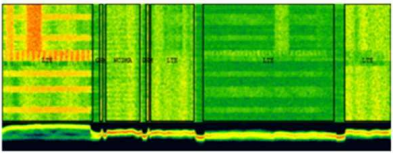
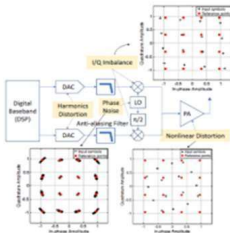
Adversarial Perturbations is a form of attack in which bad actors attempt to manipulate machine learning models that detect attacks and anomalies by “poisoning” the data they train on. These Perturbations give a understanding about the model to the attackers so that they can change the data to make the classifiers go wrong. Black box attacks, White box attacks and grey box attacks mainly target on the data to make the classifier go wrong. But the no box attacks target the meta data. We think that with experimentation and research; we could attempt to test an existing open-source dataset and model against existing no box attack or query-based attacks and to help inform the maintainers of the dangers that such techniques may face and how to mitigate them.

3.1 Impact on different trends

Impact: Commercial Trends



Impact: Academic Trends

Modulation/protocol	Emitter Identification	Detection/Coding/Channel Estimation
		
Physical layer data used: To be perturbed to deny service of classifiers		
IQ (moments, cumulants)	Impulse response, carrier phase, IQ	Spectrograms/waterfalls, channel state information, IQ

4. Previous Work

4.1 Evaluating adversarial evasion attacks in the context of wireless communication

Recent advancements in radio frequency machine learning (RFML) have demonstrated the use of raw in-phase and quadrature (IQ) samples for multiple spectrum sensing tasks. Yet, deep learning techniques have been shown, in other applications, to be vulnerable to adversarial machine learning (ML) techniques, which seek to craft small perturbations that are added to the input to cause a misclassification. The current work differentiates the threats that adversarial ML poses to RFML systems based on where the attack is executed from: direct access to classifier input, synchronously transmitted over the air (OTA), or asynchronously transmitted from a separate device.

4.1 Practical black-box attacks against machine learning

Machine learning (ML) models, e.g., deep neural networks (DNNs), are vulnerable to adversarial

examples. All existing adversarial example attacks require knowledge of either the model internals or its training data. In this paper they introduced the first practical demonstration of an attacker controlling a remotely hosted DNN with no such knowledge.

4.2 Explaining and Harnessing Adversarial Examples

Several machine learning models, including neural networks, consistently mis-classify adversarial examples. Early attempts at explaining this phenomenon focused on nonlinearity and overfitting. This paper argues instead that the primary cause of neural networks' vulnerability to adversarial perturbation is their linear nature. This explanation is supported by new quantitative results while giving the first explanation of the most intriguing fact about them.

4.3 Query-efficient hard-label black-box attack: An optimization-based approach

This paper studies the problem of attacking a machine learning model in the hard-label black-box setting. This is a very challenging problem since the direct extension of state-of-the-art white-box attacks to the hard-label black-box setting will require minimizing a non-continuous step function, which is combinatorial and cannot be solved by a gradient-based optimizer. The only current approach is based on random walk on the boundary, which requires lots of queries and lacks convergence guarantees. This work proposes a novel way to formulate the hard-label black-box attack as a real-valued optimization problem which is usually continuous and can be solved by any zeroth order optimization algorithm.

5. Research Project Plan and Approach

5.1 Research Project Phases

We plan to complete our research project in four phases. First, we will get a basic understanding of Adversarial Perturbations and fundamentals. Then we will understand about how to create a substitute model and how to craft adversarial samples. Then we try to attack the target model with these adversarial samples to misclassify and to reduce the output integrity.

Phase 1: Get a basic understanding of Adversarial Perturbations and fundamentals

Phase 2: Know how to create a substitute model and how to craft adversarial samples

Phase 3: Attack the target model with these adversarial samples

Phase 4: Once we validate our attack model, then document this study and publish using some publication framework.

5.2 Resources Required

Currently, we do not anticipate the use of any resources apart from a collection of research papers.

6. Mid Term Report

6.1 Explaining Adversarial Examples:

- $\tilde{x} = x + \eta$ where x is the input, $\tilde{x}$ is the adversarial input, η is the perturbation
- Expect same class prediction for x and $\tilde{x}$ if $\|\eta\| < \epsilon$, where ϵ is small enough to be discarded by the sensor or data storage apparatus associated with our problem.
- $w \cdot \tilde{x} = w \cdot x + w \cdot \eta$ w = weight vector, activation to grow by $w \cdot \eta$
- If w has n dimensions and the average magnitude of an element of the weight vector is m , then the activation will grow by mn
- Shows that a simple linear model can have adversarial examples if its input has sufficient dimensionality.
- Neural networks linear behavior suggests that cheap, analytical perturbations of a linear model should also damage neural networks.
- Adversarial example generated for one model is often misclassified by other models

- It is seen that multiple classifiers assign the same class to adversarial examples.
- Adversarial examples occur in contiguous regions of the 1-D subspace defined by the fast gradient sign method. Different models learn similar functions when trained to perform the same task.
- The direction of perturbation, rather than the specific point in space, matters most.
- Because it is the direction that matters most, adversarial perturbations generalize across different clean examples.

6.2 Researched on different attack techniques:

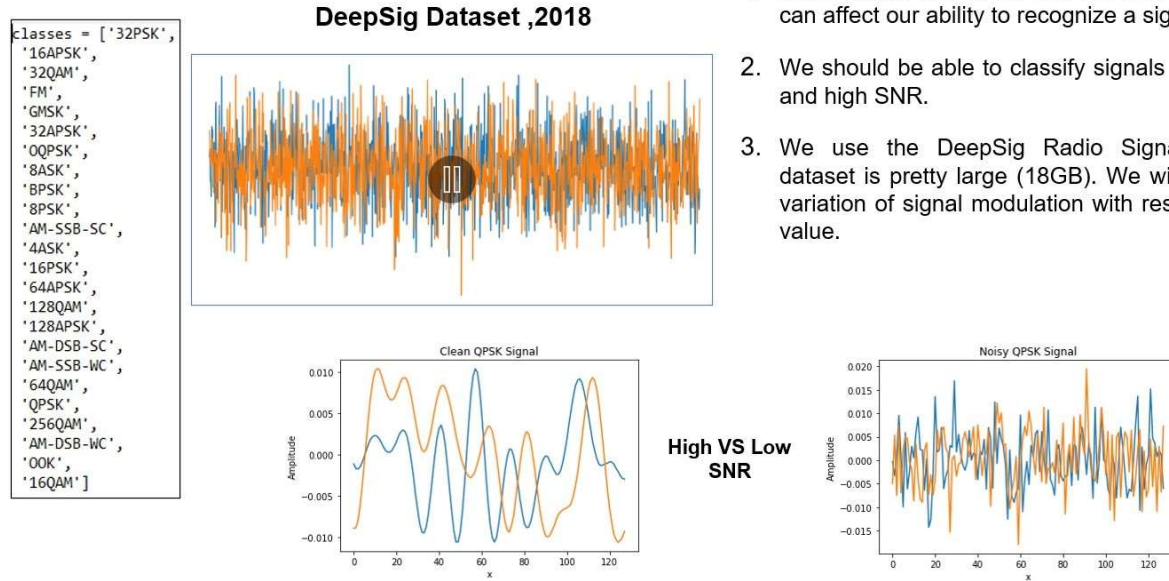
While researching and understanding about adversarial we came across two very interesting facts: First, an example generated for one model is often misclassified by other models, even when they have different architectures or were trained on disjoint training sets. Second, these different models misclassify an adversarial example, they often agree with each other on its class. We see that adversarial examples occur in contiguous regions of the 1-D subspace defined by the fast gradient sign method, not in fine pockets.

This explains why adversarial examples are abundant and why an example misclassified by one classifier has a high prior probability of being misclassified by another classifier. To explain why multiple classifiers assign the same class to adversarial examples, we hypothesize that neural networks trained with current methodologies all resemble the linear classifier learned on the same training set. This reference classifier can learn approximately the same classification weights when trained on different subsets of the training set, simply because machine learning algorithms are able to generalize. The stability of the underlying classification weights in turn results in the stability of adversarial examples. The generalization of adversarial examples across different models can be explained because of adversarial perturbations being highly aligned with the weight vectors of a model, and different models learning similar functions when trained to perform the same task. The direction of perturbation, rather than the specific point in space, matters most. Space is not full of pockets of adversarial examples that finely tile the reals like the rational numbers because it is the direction that matters most, adversarial perturbations generalize across different clean examples. We identified 2 main attacks:

1. LBFGS In target problem, L-BFGS Attack calculated approximate values of adversarial examples by line-searching with $c > 0$. We repeatedly solve this optimization problem for multiple values of c , adaptively updating c .
2. FGSM attack is a white-box attack with the goal of misclassification. The fast gradient sign method uses the gradient of the loss function to determine in which direction the pixel's intensity should be changed (whether it should be increased or decreased) to minimize the loss function; then, it shifts all pixels simultaneously.

Difference between them is, FGSM is designed primarily to be fast instead of producing very close adversarial examples.

6.3 Dataset Extraction and Analysis:



1. In the image below, we can see how drastically noise can affect our ability to recognize a signal.
2. We should be able to classify signals both in low SNR and high SNR.
3. We use the DeepSig Radio Signal Dataset, this dataset is pretty large (18GB). We will use 1.5 Million variation of signal modulation with respect to the SNR value.

7. Final Report

7.1 Model Analysis

i. Previous Method

Baseline ResNet Architecture [1]

Layer	Output Dimensions
Input	2 x 1024
Residual Stack	32 x 512
Residual Stack	32 x 256
Residual Stack	32 x 128
Residual Stack	32 x 64
Residual Stack	32 x 32
Residual Stack	32 x 16
FC/SeLU	128
FC/SeLU	128
FC/Softmax	24

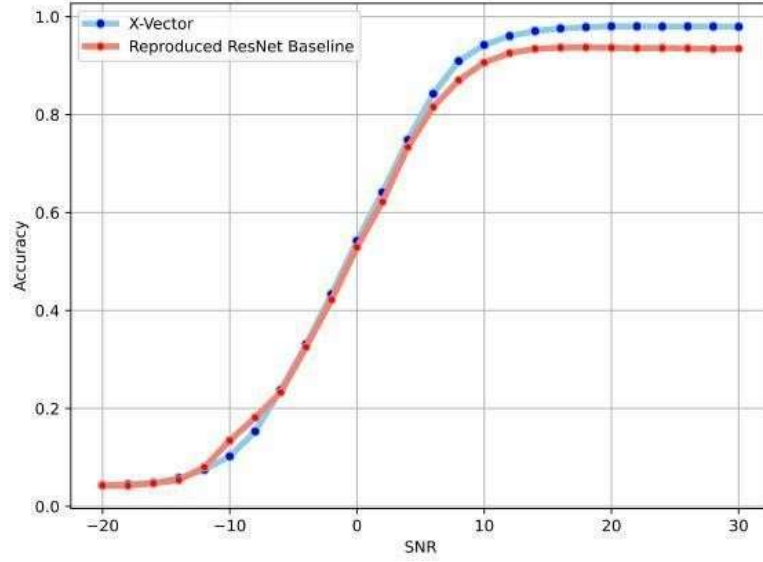
Maximum Accuracy : 93.7 %

X-Vector Architecture [2]

Layer	Output Dimensions
Input	2 x 1024
Conv 1D (ReLU)	64 x 1024
Conv 1D (ReLU)	64 x 1024
Conv 1D (ReLU)	64 x 1024
Conv 1D (ReLU)	64 x 1024
Conv 1D (ReLU)	64 x 1024
Conv 1D (ReLU)	64 x 1024
Conv 1D	64 x 1024
Average Pooling 1D	64
Variance Pooling 1D	64
Concatenate	128
FC/SeLU	128
FC/SeLU	128
FC/Softmax	24

Maximum Accuracy : 98%

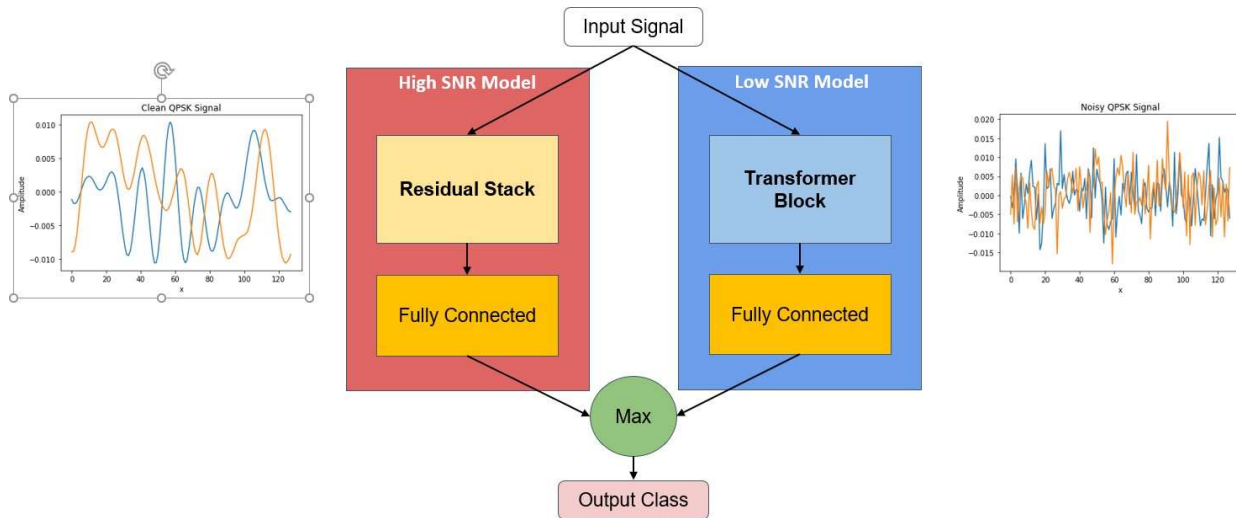
Previous method [1] uses baseline ResNet architecture and got maximum accuracy of 93.7%. New paper from the ASILOMAR Conference on Signals, Systems, and Computers in 2020 [2] got better accuracy at 98%. But we would use the first reference as our baseline since in the second reference they did not provide more detailed information about how they trained neural networks; hence we could not use their experiment as our baseline.



Baseline result compared to X-Vector result

We could see in the figure above that the result is only good for signals with high SNR, but for the signal with low SNR seems not good. From this weakness, we then proposed our method that has better accuracy in lower SNR signal but still could get a comparable result at high SNR signal.

ii. Proposed Model



Proposed method using two neural network and ensemble network

In our proposed method, we try to use two neural network models, one is good for predicting signal with high SNR and the other one is good for predicting signals with low SNR. For high SNR models, we will use a Residual Net like our baseline method and do some modification and tuning to increase its accuracy. For low SNR models, we will use a transformer neural network, this is one kind of time series neural network that could help to predict complex data for example signals with low SNR rate. After inputting our signal to two models, we will ensemble by seek the maximum accuracy output resulting from the two models.

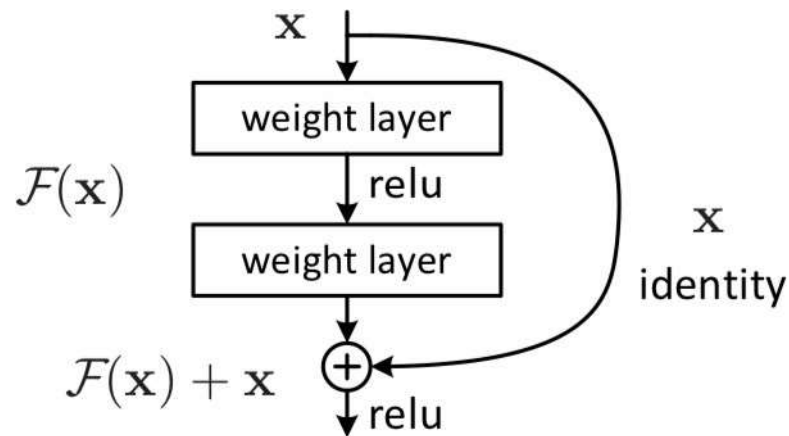
A. High SNR Model (Modified ResNet)

Residual Net Explained:

Before explaining our model, we will explain what the advantages are of using residual nets. In recent years, neural networks have become deeper, with state-of-the-art networks going from just a few layers (e.g., AlexNet) to over a hundred layers. One of the major benefits of a very deep network is that it can represent very complex functions. However, a huge barrier to training them is vanishing gradients: very deep networks often have a gradient signal that goes to zero quickly, thus making gradient descent prohibitively slow. More specifically, during gradient descent, as we backprop from the final layer back to the first layer, we are multiplying by the weight matrix on each step. If the gradients are small, due to large number of multiplications, the gradient can decrease exponentially quickly to zero (or, in rare cases, grow exponentially quickly and “explode” to take very large values).

The regular networks like VGG-16 are called “plain” networks. In plain networks, as the number of layers increases from 20 to 56 (as shown below), even after thousands of iterations, training error was worse for a 56-layer compared to a 20-layer network. In plain networks, as the number of layers increases from 20 to 56 (as shown below), even after thousands of iterations, training error was worse for a 56-layer compared to a 20-layer network.

When deeper networks are able to start converging, a degradation problem has been exposed: with the network depth increasing, accuracy gets saturated (which might be unsurprising) and then degrades rapidly. Using deeper networks is degrading the performance of the model. Microsoft Research paper tries to solve this problem using a Deep Residual learning framework.



Skip Connection in Residual Network

The approach is to add a shortcut or a skip connection that allows information to flow, well just say, more easily from one layer to the next layer, i.e., you bypass data along with normal CNN flow from one layer to the next layer after the immediate next. Two take away from the residual block, first Adding additional/new layers would not hurt the model’s performance as regularization will skip over them if those layers were not useful. Second, If the additional/new layers were useful, even with the presence of regularization, the weights or kernels of the layers will be non-zero and model performance could increase slightly. Therefore, by adding new layers, because of the “Skip connection” / “residual connection”, it is guaranteed that the performance of the model does not decrease but it could increase slightly. By stacking these ResNet blocks on top of each other, you can form a very deep network. Having ResNet blocks with the shortcut also makes it very easy for one of the blocks to learn an identity function. This means that you can stack on additional ResNet blocks with little risk of harming training set performance.

Using a very deep neural network in our works is necessary because we would classify 24 kinds of signals that represent a complex function. Using a short or small neural network will not solve the problem, this is why we need to solve our problem with a really deep neural network.

Optimizer : SGD (Baseline)

Layer	Output Dimension
Input	2 x 1024
Residual Stack	32 x 512
Residual Stack	32 x 256
Residual Stack	32 x 128
Residual Stack	32 x 64
Residual Stack	32 x 32
Residual Stack	32 x 16
FC/SeLU - Alpha Dropout	128
FC/SeLU - Alpha Dropout	128
FC/Softmax	24

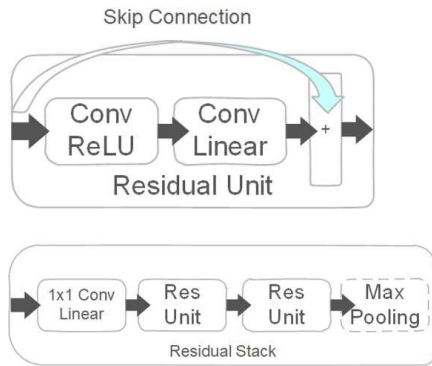
Optimizer : LazyAdam (Ours)

Layer	Output Dimension
Input	2 x 1024
Residual Stack	32 x 512
Residual Stack	32 x 256
Residual Stack	64 x 128
Residual Stack	64 x 64
Residual Stack	32 x 32
Residual Stack	32 x 16
FC/SeLU - Dropout	128
FC/SeLU - Dropout	128
FC/Softmax	24

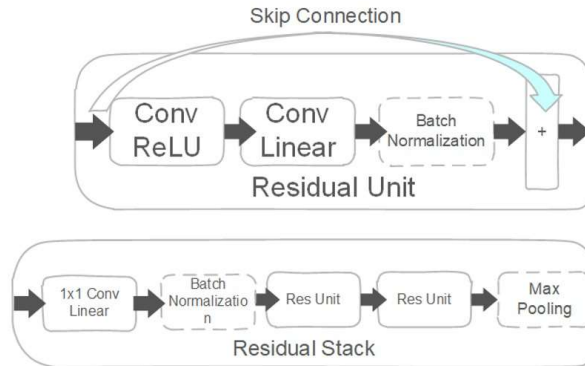
ResNet baseline vs Our ResNet

We do some modifications from the baseline to make the training become more efficient and improve its accuracy. We change the optimizer from SGD to the new state of the art of optimizer called LazyAdam. LazyAdam is a variant of the Adam optimizer that handles sparse updates more efficiently. The original Adam algorithm maintains two moving-average accumulators for each trainable variable, the accumulators are updated at every step. This class provides lazier handling of gradient updates for sparse variables. It only updates moving-average accumulators for sparse variable indices that appear in the current batch, rather than updating the accumulators for all indices. Compared with the original Adam optimizer, it can provide large improvements in model training throughput for some applications. We also increase the number of filters in the third and fourth residual stack both from 32 to 64. We then also remove two last residual stacks to improve the training split, we also remove those parts to avoid losing too much information because of skipping connection inside the residual network.

Baseline Residual Stack Composition



Our Residual Stack Composition



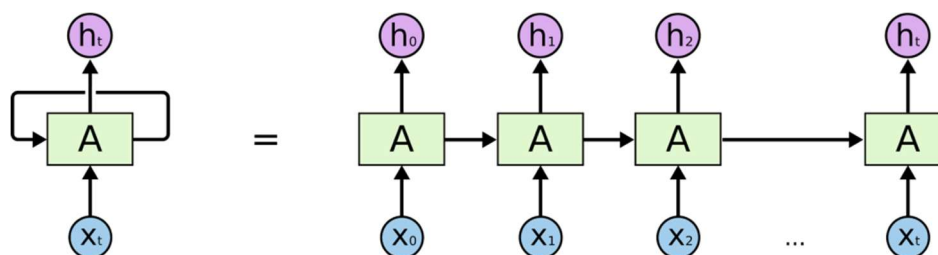
Residual Stack Baseline vs Our Residual Stack Composition

From the figure above, not only modify the neural network composition, but we also modify residual stack composition. We added batch normalization inside the residual unit, where the previous baseline not using batch normalization for their residual net. We also add batch normalization after 1x1Conv Linear inside the residual stack.

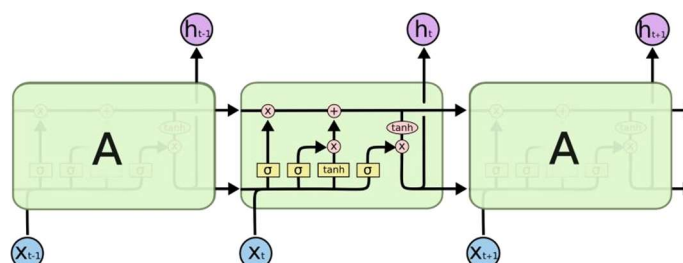
B. Low SNR Model (Transformer)

The Transformer model is the evolution of the encoder-decoder architecture, proposed in the paper titled “Attention is All You Need” [5]. While encoder-decoder architecture has been relying on recurrent neural networks (RNNs) to extract sequential information, the Transformer doesn’t use RNN. Transformer based models have primarily replaced LSTM, and it has been proved to be superior in quality for many sequence-to-sequence problems.

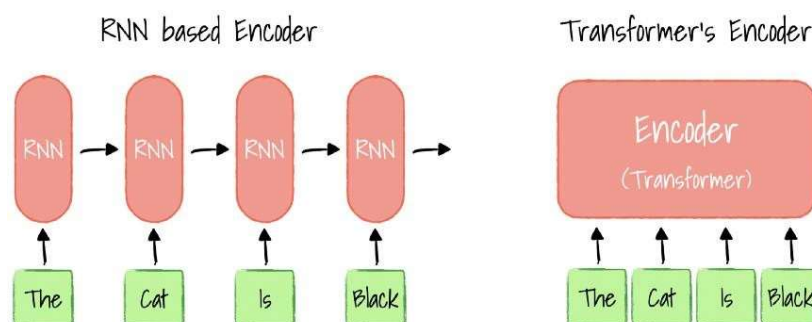
We do some research to find the best time-series model that was published recently. The first one we have known is the conventional method which uses recurrent neural networks (RNN), RNNs work like a feed-forward neural network that unrolls the input over its sequence, one after another.



This process of unrolling each symbol in the input is done by the Encoder, whose objective is to extract data from the sequential input and encode it into a vector, a representation of the input. However, RNN models have some problems, they are slow to train because it uses sequential processing instead of parallelization, and it can't deal with long sequences due to the vanishing gradient problem. The second one is the most used time-series model nowadays, which is Long-Short Term Memory (LSTM) networks, which are explicitly designed to avoid long-term dependency problems. Each LSTM cell allows past information to skip all the processing of the current cell and move to the next cell; this allows the memory to be retained longer and will enable data to flow along with it unchanged. LSTM consists of an input gate that decides what new information to be stored and a forget gate that determines what information to remove.

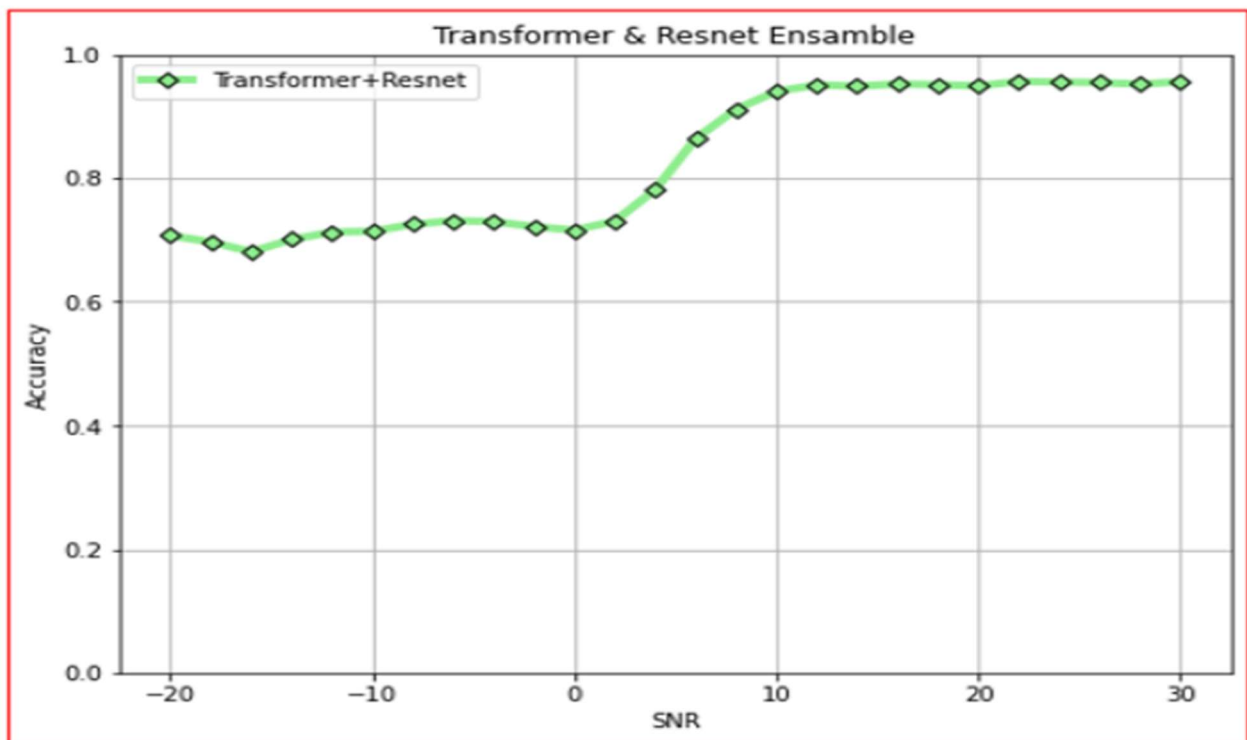
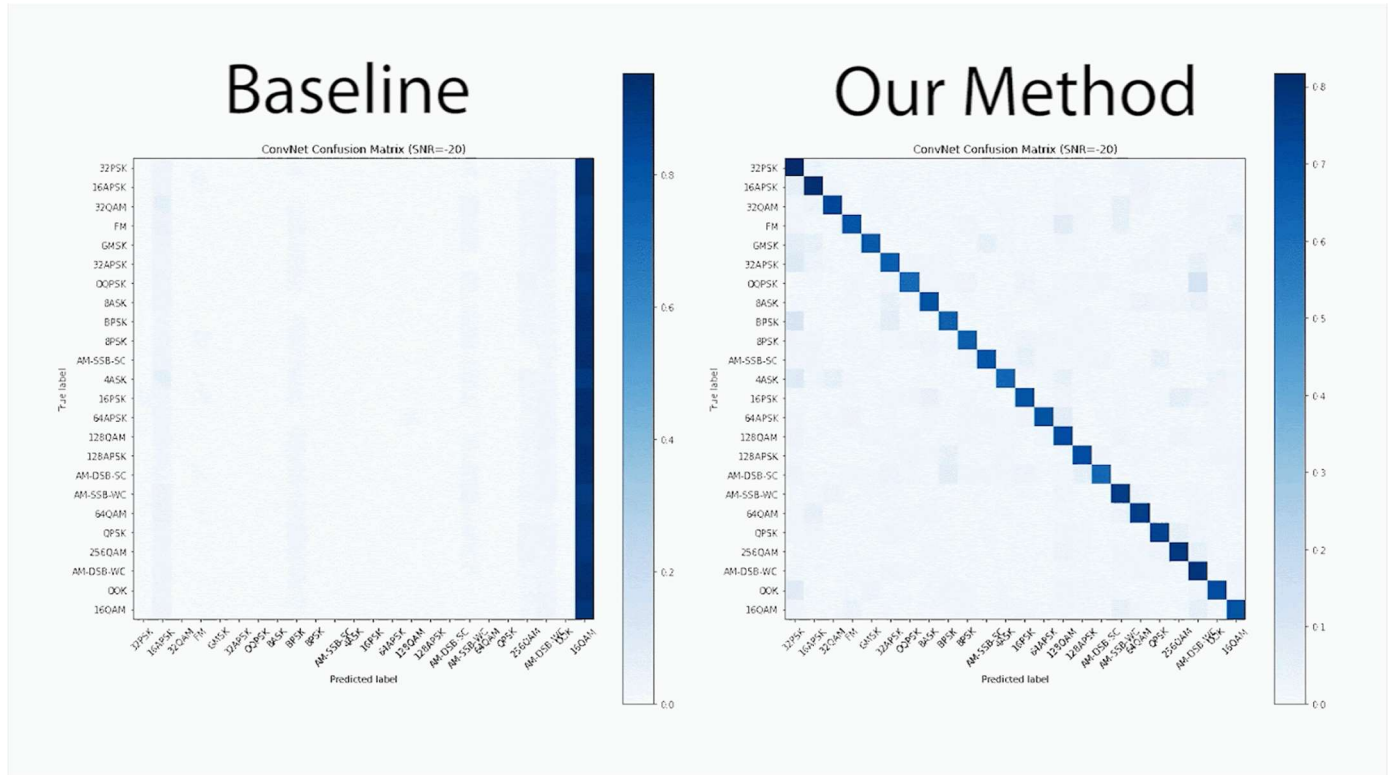


Certainly, LSTMs have improved memory, able to deal with longer sequences than RNNs. However, LSTM networks are even slower as they are more complex. Another drawback of an RNN based encoder-decoder architecture is the fixed-length vector. Using a fixed-length vector to represent the input sequence to decode an entirely new sentence is difficult. The context vector cannot store all the information if the input sequence is large. Furthermore, it is challenging to differentiate sentences with similar words but with different meanings. And then we come up with a solution using the newest state-of-the-art time series model called Transformer. It has some advantages in using this model than previous models such as it uses parallelization processing thus make use of modern graphics processing units (GPUs), which were designed for parallel computation. It can solve the vanishing gradient problem because the input size can be any size/vector length and simultaneously proceed by the network rather than sequentially. The context of data is well captured because it uses the positional encoding and self-attention mechanism which contain in the network modules.



We will explain the function of the key important network's module inside the Transformer network, more detailed explanation can refer to this [5] paper. Let's assume we are training a model that translates the English sentence to French as an easier explanation for us to understand the definition. The Transformer architecture has two parts, the Encoder (left) and the Decoder (right).

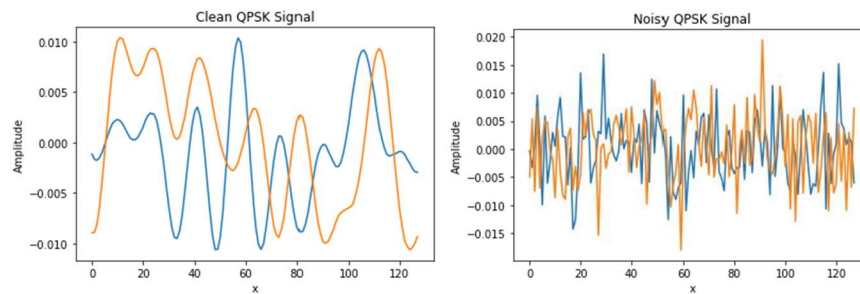
7.2 Our Model



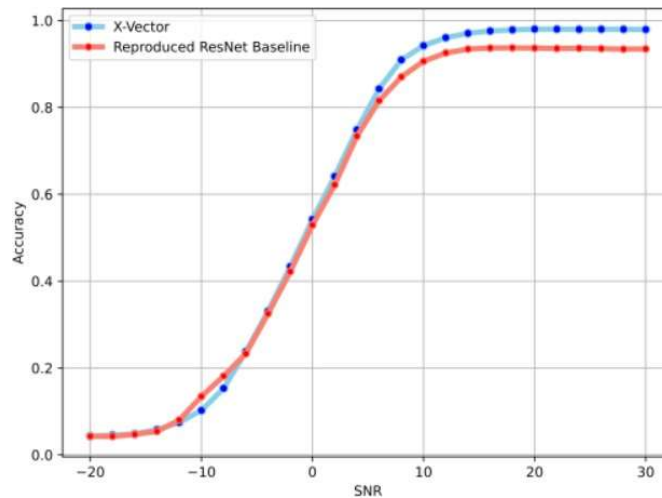
7.3 Challenges Faced

As most of us are beginners in the field of ML, tackling big data sets and understanding the models was a hurdle for us initially. We tried to learn different ML models and how the data set parameters are used to increase the accuracy of the model.

As we discussed before, In this project We have used the DeepSig RadioML Dataset, this dataset is pretty large (19GB). The dataset has approximately 1.5 million variation of signal modulation with respect to the SNR value. Due to the data set size and as we have limited resources like computation power, we tried to implement this on multiple systems. In reality, the signal is always not ideal and combined with the unwanted signal that is considered as noise. In the image above, we can see how drastically noise can affect our ability to recognize a signal.



A clean signal will have a high SNR and a noisy signal will have a low SNR. We should be able to classify signals both in low SNR and high SNR using the RAW signal without preprocessing needed. Denoising or preprocessing many kinds of signal is not an easy task, because some signal needs different treatment in case of preprocessing, sometimes we could not do the preprocessing to the signal that we do not know what the signal is. Hence, one of the challenges is how we could classify the RAW signal directly.



We use the DeepSig Radio Signal Dataset, this dataset is pretty large (18GB) with consists 24 types of signal modulation such as 32PSK, 16APSK, 32QAM, FM, GMSK, 32APSK, OQPSK, 8ASK, BPSK, 8PSK, AM-SSB-SC, 4ASK, 16PSK, 64APSK, 128QAM, 128APSK, AM-DSB-SC, AM-SSB-WC, 64QAM, QPSK, 256QAM, AM-DSB-WC, OOK, and 16QAM. We need to determine what is the best network that could classify both low and high SNR signals faster.

The concept of Adversarial Perturbations is relatively new. So, we tried to get an in-depth knowledge of how it's done and what it can do to the accuracy of the model. As a part of this study, we were able to modify an existing algorithm which will allow us to classify signals both in low SNR and high SNR.

As we extracted the data and modified the data, we tried to learn how to create a substitute model and reduce the

targets model accuracy. We were able to create and train a model. As the dataset is huge, running it on our systems with limited resource took almost 6 hours for each Epoch. An epoch is when all the training data is used at once for training the machine learning model. There were 5 Epochs in this model we proposed. We face difficulty here as the system would crash in between and we need to start again from the beginning. To overcome this problem, we saved the model right after it's execution and used the saved model in the next phase of project. After creating and training the model we tried to perform the attack with parameters from the model, here we ran into a problem where the system would not allow TensorFlow module to use the GPU resources on the system and which always resulted in dead kernel.

We tried multiple resources to overcome this error, but our effort was inconclusive.

8. Conclusion and Future Work

Finally, I would like to conclude by saying that were able to extract the data and modify the data in such a way that we can classify signals that have high SNR values and low SNR values. We were also able to create a model and train the model in such a way it eliminates multiple false scenarios. We have identified few particular scenarios and documented them so that they may not be repeated again to overcome the errors in future. We were not able to test the attack on the model as we were tagged with the max pooling error and kernel being dead.

As a future work we might do an efficient attack on the model to change the accuracy and overcome the errors.

9. Investigator Biographies

Raviteja Avuthu holds a bachelors and master's degree in Computer Science and is doing his second masters at The University of Texas at Dallas, graduating in December 2023 with a focus in Data Science. He has experience working in IBM as a Software Developer. He has done Machine Learning as an undergraduate course and is looking forward to gaining expertise in the field of Data Science. He also has experience in Python and R.

Raghuveer Pachipulusu Graduated with BTech in Computer Science and currently pursuing master's in computer science at University of Texas at Dallas on Data Science Track. He Has Previously worked on Machine Learning and Deep Learning Projects and Published a Technical Paper in International Conference. He Posses Web Development Skills as well as Programming skills such as Python, Java, SQL, and R. He has taken courses in Cryptography and Network Security

Saipriya Nimmagadda holds a bachelor's degree in Computer Science and is pursuing her masters at The University of Texas at Dallas, graduating in May 2023 with a focus in Data Science and cloud computing. She started her career in Cerner healthcare Pvt Ltd as a Software Engineer with 2+ years of experience. She is a certified AWS cloud practitioner who is currently working as AWS developer for office of information technology at university of Texas at Dallas. Gaining more exposure in cloud and machine learning areas is her main goal.

Srinivas Panchangam holds a bachelor's degree in Electronic and Communications Engineering and is pursuing his masters at The University of Texas at Dallas, graduating in Dec 2023 with a focus in Data Science. He started his career as a software developer in one of the biggest tech companies in India, TCS. Though being from a different bachelors background his interest towards data science never ended, so he is trying to explore the field and get expertise in that.

With a bachelor's in computer science and an associate in management, **Varun Potluri** is a second- year master's student in computer science specializing in cyber security. He has experience working in a variety of fields, such as health care, risk management, software development, mergers and acquisitions, cloud security using Amazon Web Services, and cyber security. During his bachelor's, Varun published 2 research papers in cyber security space with an aim to increase the security of storage application using blockchain technology. Mr. Potluri was very interested in programming, reverse engineering, malware analysis, pen testing, risk management, and cloud security throughout his security-related studies.

10. Bibliography

[1] Flowers, Bryse, R. Michael Buehrer, and William C. Headley. "Evaluating adversarial evasion attacks in the context of wireless communications." *IEEE Transactions on Information Forensics and Security* 15 (2019): 1102-1113.

[2] Practical black-box attacks against machine learning.
<https://dl.acm.org/doi/pdf/10.1145/3052973.3053009?ref=https://githubhelp.com>

[3] Explaining and Harnessing Adversarial Examples. <https://arxiv.org/pdf/1412.6572.pdf>

[4] Query-efficient hard-label black-box attack: An optimization-based approach